

CSE 144 Transfer Learning Challenge

Jason Vu 1975992

Dominic Marella 1929850

Cagn Steinbrecher 1968375

1 Introduction

At its core, this is an image classification challenge with 100 distinct classes, where we aim to develop a model that classifies images using a training pipeline. Transfer learning is a good approach for this problem, given the relatively small sample size. By using a pre-trained model, we can leverage prior knowledge from other image recognition tasks for this task at hand. Our methodology involves a two-stage process: an initial Optuna hyperparameter search on a single holdout split, followed by 4-fold cross-validation for the top four-performing trials. We also use a couple of data augmentation techniques, namely Mixup and Cutmix, to improve the model's generalization capabilities. The final predictions are then generated from an ensemble of the best trial's 4-fold checkpoints, yielding a yield of 0.87118.

2 Dataset

Number of classes and dataset sizes (train/val/test)

In terms of classes and dataset sizes, we had:

- 100 classes
- 1079 training images
- 1036 training images

For validation purposes, the validation set was dynamically created for each fold during training. For Optuna tuning, a single validation set was created, comprising 25% of the training data. We then performed a 4-fold cross-validation procedure, splitting the data into 4 folds and using one fold for validation at each iteration[1].

2. Directory structure and label mapping details (ensure labels 0–99 match folders).

The training data was located in the directory:

- /content/drive/MyDrive/Kaggle/train

The test data directory was located in:

- /content/drive/MyDrive/Kaggle/test

For label mapping, the labels (0-99) were derived from the directory names in the training directory, with each subdirectory corresponding to a class, and image files within said subdirectories are assigned the proper class label.

3. Preprocessing (resize, normalization) and data augmentation used.

For preprocessing the data, we converted the images to RGB and normalized them using ImageNet[2]. The mean values for Imagenet were 0.485 (red pixel density), 0.456 (green pixel density), and 0.406 (blue pixel density), and the Imagenet standard deviation values were 0.229 (red pixel density), 0.224 (green pixel density), and 0.225 (blue pixel density). This allowed us to stabilize model training and ensure compatibility with the model's pre-trained state, since it expects inputs within a specific range.

To prevent overfitting, we used a variety of data augmentation techniques, applying them only to the training images. The steps we took included randomly cropping and resizing images in order to force the model to recognize objects at different scales and positions[3]. We also randomly flipped images horizontally and rotated them to introduce geometric variation[4]. Another step we took was to manipulate various color properties in the images, such as brightness, contrast, saturation, and hue[5]. This would allow the model to be less dependent on lighting conditions to make accurate assumptions about the images. Finally, we utilized more advanced techniques, such as Mixup and Cutmix, which blend images and their labels to create new examples, further honing the model's generalization abilities[6, 7].

3 Implementation

3.1 Model

The model we chose was a pre-trained EVA02-Large vision transformer backbone[8]. We selected this model because, among the models we tested beforehand, it achieved the best results. For the 100-class classification task, the original classifier head was adapted to output 100 classes. We also applied dropout to the new head, and stochastic depth was used within the transformer blocks. Our fine-tuning approach involved training the entire model from scratch,

updating all layers during training without freezing any, so the model can fully adapt to our dataset.

3.2 Training

For the loss function during training, the model uses SoftTargetCrossEntropy; during evaluation, the loss function used is nn.CrossEntropyLoss[9]. The optimizer we used was AdamW[10]. For key training parameters like during trial 18 with the 4 fold cross validation run included a learning rate of 5.0×10^{-5} , a cosine annealing like learning rate scheduler with a 3 epoch warm up phase with an effective batch size of 42 that had an accumulation step of 1. Training consisted of 200 epochs with a weight decay of 0.0465. The software environment was set up for utilizing CUDA GPUs, we were auto assigned a Nvidia RTX Pro 6000 Blackwell due to unavailability issues. The prediction stages used an Nvidia L4 GPU. Both cards were accessed through a Google Colab Pro subscription. The PyTorch version we used was version 2.11.0+cu128[11].

4 Experiments

4.1 Baseline Setup

For the baseline, I used a pre-trained EVA02-L model at 448×448 image resolution. The dataset was organized into training and test folders for final prediction. I checked to make sure that all 100 classes and training images were loaded. The training setup used AdamW for hyperparameter tuning and imagenet normalization to match the pretrained model's normalization style. The Learning rate is approximately 5×10^{-5} . During training, I used Soft target cross entropy because Mixup and Cutmix produce soft labels rather than single hard class labels, and for validation/test, I used standard cross entropy loss because validation images are evaluated with their original hard labels. The model was then finetuned for 100-class image classification. Training and validation accuracy were recorded for each epoch in a JSON file, and the best checkpoint for each fold was saved based on validation accuracy, with the final output in ID and target format, as required by the assignment.

4.2 Hyperparameter Tuning Plan and Validation Method

Before the final full run, I performed hyperparameter tuning and out of the 40 different sets of hyper parameters I chose trial 18 as the best configuration. Trial 18 initially had an accuracy of

0.9000 so I reused those specific hyper parameters for folding. Because I used Cutmix/Mixup and soft target entropy as my loss function the training curve is much more chaotic but did not overfit the data.

For validation, I used 4-fold validation meaning each fold is trained on 75% of the data and validated on the remaining 25%. After all folds were trained, I averaged the best validation accuracy as my mean validation accuracy and used each fold as part of an ensemble for test prediction with each fold having 25% weight in the final probability.

4.3 Ablation

For image transformations, I used several data augmentation methods to make the model more generalizable and reduce overfitting. The model could potentially memorize exact images rather than general visual patterns, so I used random resized crops, horizontal flips, small random rotations, and color jitter to introduce variation and increase generalizability.

For sample mixing, I used MixUp and CutMix as stronger regularization techniques to blend images and labels, reducing the model's tendency to memorize individual training images. These techniques make the classification task harder because models can't memorize images.

For model size, I used EVA02-L, which had a strong backbone. I wanted to train the full backbone to update internal image features to better fit the problem at hand and used strong regularization to control overfitting and fine-tune the whole model.

5 Results

5.1 Training/Validation Accuracy

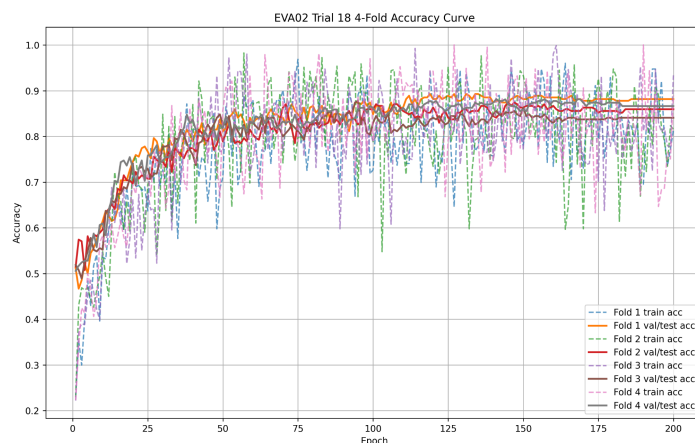


Fig. 1 Accuracy Score for EVA02: dotted lines showcase the training accuracy and full lines show the validation/test accuracy

Figure 1 shows the accuracy scores for both validation and training across all 4 folds. When looking at this, the training set is more unstable because we used Mixup, Cutmix, and soft target cross entropy instead of the regular cross entropy for evaluating training accuracy. However, when switching back to cross-entropy, we find less instability. We clearly see that the four folds have approximately the same accuracy, which strengthens our confidence in our results.

5.2 Training/Validation Loss

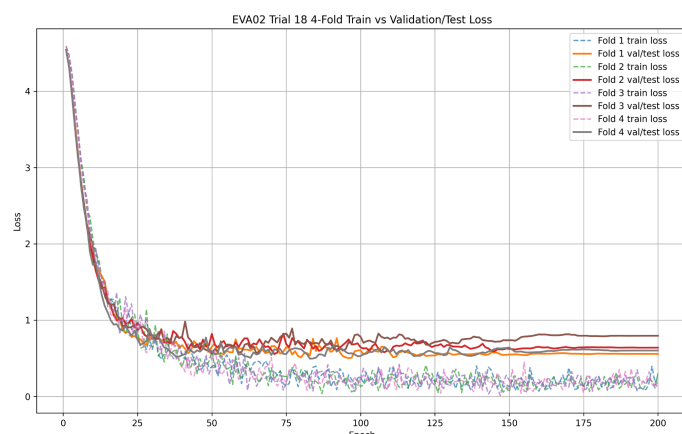


Fig 2 Loss score for EVA02: dotted lines showcase the training loss and full lines show the validation/test loss

Figure 2 shows the loss for both the training and validation of all 4 folds. We see a higher loss in the validation set compared to the training set, potentially due to different metrics at play: the training set uses soft label probabilities to determine accuracy, while the validation set uses hard labels. It confirms that my training metric is what is holding it back because it gets much closer to 0 than my validation set.

5.3 Kaggle Public Leaderboard Score

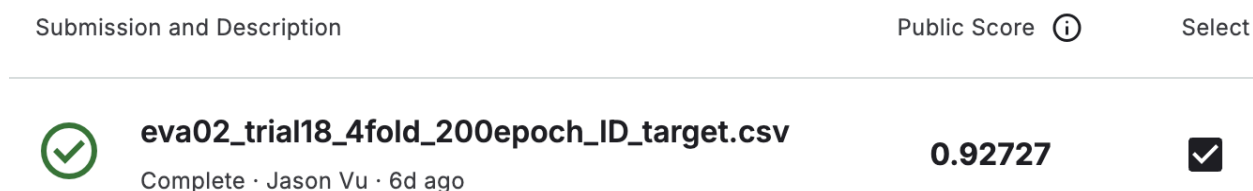


Fig. 3 Kaggle public score at 92.72% submitted by Jason Vu, where the left column is ID in the form of .jpg and the right column is the target

Figure 3 shows my Kaggle public leaderboard score, showcasing my results after 4-fold of eva02, which was higher than my average validation score and also higher than my 4-TTA run. Because the public data targets only 11% of the entire test set, I hypothesize that the score will decrease to around 88%, which was our average accuracy score after 4 folds.

6 Discussion

6.1 What worked best and why.

There are several components that worked best for this experiment in terms of what could have been utilized. The general oversight is the heavy reliance on different pretrained models in this experiment. This allowed us to rapidly build our experiment and achieve early success in our testing scores. With early usage, we were able to build on the different models and ensembles to determine which platforms we liked best.

One such selection would be the usage of the EVA-02 model in our experiment. This model is a transformer-based visual representation model that leverages multiple aspects of image processing and construction. Perhaps the most important aspect of this model with regard to the transfer learning challenge is its ability to rebuild the unknown and obscured parts of different target pictures.

Furthermore, the practice of checkpointing during the experiment enhances its effectiveness. Checkpointing allowed the results of each epoch to be saved and the highest scores to be recorded for use. Early stopping is triggered when validation data stops improving, and training begins to overfit.

Finally, extensive regulatory measures within the model significantly increase its efficacy. Because of the large size of our chosen model and the relatively small sizes of the test and train datasets, our model must incorporate several regularization methods. This includes, but is not limited to, many of our hyperparameters such as weight decay, drop rate, drop path rate, mixup alpha, cutmix alpha, and color strength.

6.2 Failure cases, overfitting/underfitting observations.

Considering the early models of our assignment, there were many failure cases and observations of both overfitting and underfitting. The most concerning failure case lies in memory management. With the usage of A100 and T4 GPUs, there were early on memory management failures and issues with Google Drive mounting and memory manipulation. There was also a chance of hyperparameter erasure, in which certain hyperparameters were prematurely removed from the model if deemed inefficient early on.

Overfitting and underfitting also posed a key hurdle to early versions of our model. Nearly all of this can be attributed to hyperparameter regularization and erasure. Two main components that

led to overfitting early on were the sheer size of our model, when combined with the much more limited data set. This allowed for complex machinations on a much smaller set of fixed data, which, among many iterations, led to overfitting. A lack of regulation or inefficient hyperparameter settings also led to model overfitting. At the same time, underfitting occurred when hyperparameters were too strictly controlled or when epochs lacked sufficient practice iterations. This showed lower scores in both the training and test environments, which is a clear sign of underfitting.

6.3 Limitations and concrete next improvements.

The current model's limitations mainly stem from hyperparameter settings. With a large number of hyperparameters, a single change in a setting value can lead to drastic changes in regularization, which in turn can spike overfitting and underfitting in the model.

Google Drive memory allocation is also another concern, but this can be remedied with subscriptions. Obviously, this is not optimal, so developments in storage management are definitely concrete next steps. The model itself is very large and runs through the training and test data throughout an extensive number of epochs. This can also create hardware complications for memory and processing units. This mainly has to do with hardcoding throughout our model, which will also be an improvement in the next version.

Finally, we would like to implement DINOv3 as the next model to assist with image production and processing[12]. This model is available through cooperation with PyTorch Meta AI. It mainly concerns advanced data recognition and image reconstruction within a model and platform with data and memory constraints, enabling several processing improvements within our model.

7 Reproducibility

7.1 Random seeds and determinism settings

Random seeding and determinism settings are set 3 times throughout the experiment, once in the Optuna trial stage, once in Trial 18, where the highest validation score was recorded, and then finally in the 4-fold validation stage/4 TTA/test prediction stage. The global seed for each stage is set to 42, and the `seed_everything(SEED)` global function is used to ensure reproducibility across different Python packages.

Within the Optuna trial stage, the experiment utilizes code `seed_everything(SEED + trial_number * 100 + fold)` so that each Optuna trial with respect to cross-validation folds retains

reproducibility with random seeding. Within the CUDA section of this stage, several lines of code set the determinism through `torch.backends.cuda.matmul.allow_tf32 = True`, `torch.backends.cudnn.allow_tf32 = True` and `torch.backends.cudnn.benchmark = True`. These lines significantly improve determinism within the experiment while retaining reproducible results.

Within Trial 18, `seed_everything(SEED + trial_number * 100 + fold)` is called within the `train_one_fold` function to account for the consistent nature of the trial and allow for reproducibility among each fold. In the CUDA section of this stage, the same determinism settings are used.

Within the CUDA section of the 4-fold validation stage/ 4TTA/test prediction stage, the exact same determinism settings are utilized.

7.2 Package versions / environment setup steps.

The package version / environment setup first installs all of the libraries that will be used: `!pip install -q timm scikit-learn matplotlib tqdm pandas pillow`:

Next, Google Colab mounts the Google Drive Kaggle folder which contains the necessary training and test data. It allows for ease of access and submission files. `!from google.colab import drive drive.mount('/content/drive')`:

After this, the operating system environment is setup as :

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
```

```
os.environ["TOKENIZERS_PARALLELISM"] = "false"
```

These lines assist in GPU data management and turn off tokenizers parallelism in order to stop errors in the complexity of the GPU runtime operations.

Finally, the os calls for imports, checkpointing, and storage commands:

```
import os
```

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
```

```
os.environ["TOKENIZERS_PARALLELISM"] = "false"
```

```
import gc
```

```
import json
```

```
import math
```

```
import time
```

```
import random
```

```
import shutil
```



```

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
from PIL import Image

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torch.amp import autocast, GradScaler

import timm
import optuna
from optuna.samplers import TPESampler
from timm.data import Mixup
from timm.loss import SoftTargetCrossEntropy

from torchvision import transforms
from sklearn.model_selection
import StratifiedKFold, StratifiedShuffleSplit

try:
    from google.colab import drive
    drive.mount("/content/drive")
except Exception:
    pass

# =====
# PATHS
# =====

TRAIN_DIR = "/content/drive/MyDrive/Kaggle/train"
TEST_DIR = "/content/drive/MyDrive/Kaggle/test"

# Drive output directory: only small metadata + final outputs + copied
# top4 trial checkpoints go here.
SAVE_DIR =
"/content/drive/MyDrive/Kaggle/eva02_h100_40trials_tunelfold_top4_4fold"
DRIVE_TRIALS_DIR = os.path.join(SAVE_DIR, "top4_fullcv_checkpoints")

```

```

# Fast local directory: ALL trial checkpoints are saved here first.
# In Colab/Kaggle this is much faster than writing every .pt directly to
Drive.
LOCAL_WORK_DIR = "/content/eva02_h100_40trials_tunelfold_top4_4fold"
LOCAL_TRIALS_DIR = os.path.join(LOCAL_WORK_DIR, "all_trial_checkpoints")

os.makedirs(SAVE_DIR, exist_ok=True)
os.makedirs(DRIVE_TRIALS_DIR, exist_ok=True)
os.makedirs(LOCAL_WORK_DIR, exist_ok=True)
os.makedirs(LOCAL_TRIALS_DIR, exist_ok=True)

FINAL_CSV_PATH = os.path.join(SAVE_DIR,
"eva02_h100_best4trials_ID_target.csv")
FINAL_PROBS_PATH = os.path.join(SAVE_DIR,
"eva02_h100_best4trials_probs.npy")
INFO_PATH = os.path.join(SAVE_DIR, "study_best_trials.json")
TRIALS_CSV_PATH = os.path.join(SAVE_DIR, "optuna_trials.csv")
BEST4_CKPT_CSV_PATH = os.path.join(SAVE_DIR,
"best4_trial_fold_checkpoints.csv")

# Keep the Optuna DB on Drive so you can resume after disconnects.
# Checkpoints stay local until a trial enters the current top 4.
STUDY_DB_PATH = os.path.join(SAVE_DIR, "eva02_h100_optuna_study.db")

print("Drive save dir:", SAVE_DIR)
print("Local checkpoint dir:", LOCAL_TRIALS_DIR)
print("Drive top-4 checkpoint dir:", DRIVE_TRIALS_DIR)
print("Two-stage mode: all trials use 1 holdout split; only top 4 use
4-fold CV")
print("RAM-safe DataLoader: NUM_WORKERS=0, PIN_MEMORY=False,
PRED_BATCH=8")

```

7.3 Exact commands to train and to generate submission.csv.

Within Optuna Trial stage

```

# Trains the initial 40 tuning trials
study.optimize(objective, n_trials=N_TRIALS, callbacks=[trial_callback],
gc_after_trial=True)

# Trains the full 4-fold CV on the top 4 trials

```

```
fullcv_summary = run_full_cv_for_top_trials(study)
```

Within Trial 18 stage

```
for fold, (train_idx, val_idx) in  
enumerate(skf.split(np.zeros(len(labels)), labels), start=1):  
    fold_result = train_one_fold(  
        trial_number=FIXED_TRIAL_NUMBER,  
        fold=fold,  
        train_idx=train_idx,  
        val_idx=val_idx,  
        p=p,  
        trial_dir=LOCAL_CKPT_DIR,  
    )  
# Generates predictions and saves the CSV  
submission, ensemble_probs = predict_4fold_ensemble(fold_results)
```

Within 4-fold validation stage/10 TTA/test prediction stage

```
submission = pd.DataFrame({  
    "ID": test_ids_final,  
    "target": test_preds,  
})  
  
submission.to_csv(OUT_CSV, index=False)  
Click run all
```

8 Team Contributions

Member 1: Jason Vu

- Testing of Convnext, Resnet, Swine, and Eva02
- Fine tuning of final model
- Running Tests, Trials and Final 4 Fold 4-TTA model
- Hyperparameter selection

Member 2: Dominic Marella

- Refinement of final model
- Experimentation with different streamlined EVA-02 modelings
- Formulation, debugging, and model comparison of chosen final model with a renovated experimental G4 Ensemble provided from original group work tuned for accuracy but ultimately abandoned for memory/Google Drive and time limit concerns. Early sampling provided the following score before early stopping

```
eva02 | Fold 4 | Epoch 117/120 | TrainAcc 0.7063 | ValAcc 0.8810 |  
EmaAcc 0.7472 | Gap -0.1748 | Best 0.8885@97 | Bad 20/20 | 31.0s  
Early stopping.
```

3. Member 3: Cagn Steinbrecher

- Model refinement
- Code organization

9 References

1. TorchVision documentation / pretrained model sources.

NOTE OUR PRETRAINED MODELS - REFERENCED IN DISCUSSION

- [1] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). *Optuna: A next-generation hyperparameter optimization framework*. arXiv. <https://doi.org/10.48550/arXiv.1907.10902>
- [2] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255. <https://doi.org/10.1109/CVPR.2009.5206848>
- [3] Takahashi, R., Matsubara, T., & Uehara, K. (2019). Data augmentation using random image cropping and patching for deep CNNs. *IEEE Transactions on Circuits and Systems for Video Technology*. <https://doi.org/10.48550/arXiv.1811.09030>
- [4] Hewapathirana, K. (2025, April 11). *Image resizing and normalization in machine learning*. Medium. <https://medium.com/@kalanarandil/image-resizing-and-normalization-in-machine-learning-f461d4e96b07>
- [5] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25. <https://proceedings.neurips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>
- [6] Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond empirical risk minimization. *International Conference on Learning Representations*. <https://arxiv.org/abs/1710.09412>
- [7] Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J., & Yoo, Y. (2019). CutMix: Regularization strategy to train strong classifiers with localizable features. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 6022–6031. <https://doi.org/10.1109/ICCV.2019.00612>
- [8] Fang, Y., Sun, Q., Wang, X., Huang, T., Wang, X., & Cao, Y. (2024). EVA-02: A visual representation for neon genesis. *Image and Vision Computing*, 149, Article 105171. <https://doi.org/10.1016/j.imavis.2024.105171>
- [9] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- [10] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*. <https://arxiv.org/abs/1711.05101>
- [11] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32. <https://arxiv.org/abs/1912.01703>

[12] Siméoni, O., Vo, H. V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S., Ramamonjisoa, M., Massa, F., Haziza, D., Wehrstedt, L., Wang, J., Darcet, T., Moutakanni, T., Sentana, L., Roberts, C., Vedaldi, A., ... Bojanowski, P. (2025). *DINOv3*. arXiv. <https://doi.org/10.48550/arXiv.2508.10104>